MLFP mlfp05 — Speaker Notes

- Module 5: Deep Learning and ML Mastery in Vision and Transfer Learning — Speaker Notes
  - Slide 1: Module 5 — Title
  - Slide 2: What You Will Learn
  - Slide 3: Your Journey — 8 Lessons
  - Slide 4: Kailash Engines in Module 5
  - Slide 5: DL Toolkit Refresher (from M4)
  - Slide 6: Lesson 5.1 — Autoencoders (Title)
  - Slide 7: The Autoencoder Architecture
  - Slide 8: Reconstruction Loss
  - Slide 9: Variant 1 — Vanilla Autoencoder
  - Slide 10: Variant 2 — Denoising Autoencoder (DAE)
  - Slide 11: Variant 3 — Variational Autoencoder (VAE)
  - Slide 12: VAE Loss — The ELBO
  - Slide 13: The Reparameterisation Trick
  - Slide 14: Variant 4 — Convolutional Autoencoder
  - Slide 15: Additional Autoencoder Variants (Survey)
  - Slide 16: Exercise 5.1 — Autoencoder Workshop
  - Slide 17: Lesson 5.1 Summary
  - Slide 18: Lesson 5.2 — CNNs and Computer Vision (Title)
  - Slide 19: The Convolution Operation
  - Slide 20: Pooling and Feature Hierarchy
  - Slide 21: CNN Architecture Evolution
  - Slide 22: ResNet — Skip Connections
  - Slide 23: Modern Enhancement — SE Blocks
  - Slide 24: Modern Training Enhancements
  - Slide 25: Vision Transformers (ViT) — Brief Introduction
  - Slide 26: Kailash Bridge — OnnxBridge
  - Slide 27: Exercise 5.2 — CNN Classification Pipeline
  - Slide 28: Lesson 5.2 Key Formulas
  - Slide 29: Lesson 5.2 Summary
  - Slide 30: Lesson 5.3 — RNNs and Sequence Models (Title)
  - Slide 31: Recurrent Neural Networks
  - Slide 32: LSTM — Long Short-Term Memory
  - Slide 33: LSTM — All Five Equations
  - Slide 34: LSTM — Output Gate and Hidden State
  - Slide 35: GRU — Gated Recurrent Unit
  - Slide 36: Attention Mechanisms for Sequences
  - Slide 37: Sequence Model Metrics
  - Slide 38: Applications — Finance and Text
  - Slide 39: Exercise 5.3 — Sequence Modelling
  - Slide 40: Lesson 5.3 Summary
  - Slide 41: Lesson 5.4 — Transformers (Title)
  - Slide 42: Self-Attention — The Core Idea
  - Slide 43: Scaled Dot-Product Attention
  - Slide 44: Why Divide by sqrt(d_k)?
  - Slide 45: Multi-Head Attention
  - Slide 46: Positional Encoding
  - Slide 47: Transformer Architecture
  - Slide 48: Transformer Variants
  - Slide 49: Consolidation — Four Paradigms Compared

- Slide 50: Exercise 5.4 — Transformers
- Slide 51: Lesson 5.4 Key Formulas
- Slide 52: Lesson 5.4 Summary
- Slide 53: Lesson 5.5 — GANs and Diffusion (Title)
- Slide 54: GAN — Generator vs Discriminator
- Slide 55: GAN Minimax Objective
- Slide 56: DCGAN — Deep Convolutional GAN
- Slide 57: WGAN — Wasserstein GAN
- Slide 58: GAN Variants (Survey)
- Slide 59: Diffusion Models (Brief)
- Slide 60: GAN Training Challenges
- Slide 61: Exercise 5.5 — Generative Models
- Slide 62: Lesson 5.5 Summary
- Slide 63: Lesson 5.6 — Graph Neural Networks (Title)
- Slide 64: Graph Data — Nodes, Edges, Adjacency
- Slide 65: GCN — Graph Convolutional Networks
- Slide 66: GraphSAGE and GAT
- Slide 67: GNN Task Types
- Slide 68: Exercise 5.6 — Graph Neural Networks
- Slide 69: Lesson 5.6 Summary
- Slide 70: Lesson 5.7 — Transfer Learning (Title)
- Slide 71: The Transfer Learning Paradigm
- Slide 72: CV Transfer Learning — ResNet Fine-Tuning
- Slide 73: NLP Transfer Learning — BERT Fine-Tuning
- Slide 74: Architecture Selection Guide
- Slide 75: Kailash Bridge — OnnxBridge + InferenceServer
- Slide 76: Exercise 5.7 — Transfer Learning Pipeline
- Slide 77: Lesson 5.7 Summary
- Slide 78: Lesson 5.8 — Reinforcement Learning (Title)
- Slide 79: The RL Paradigm Shift
- Slide 80: RL Fundamentals
- Slide 81: Bellman Equations
- Slide 82: DQN — Deep Q-Network
- Slide 83: Policy Gradient Methods
- Slide 84: Five Algorithms, Five Applications
- Slide 85: DDPG and SAC
- Slide 86: A2C — Advantage Actor-Critic
- Slide 87: PPO — Proximal Policy Optimization
- Slide 88: Custom Gymnasium Environments
- Slide 89: Kailash Bridge — RLTrainer
- Slide 90: Exercise 5.8 — Reinforcement Learning
- Slide 91: Lesson 5.8 Key Formulas
- Slide 92: Lesson 5.8 Summary
- Slide 93: Module 5 — Complete Formula Reference
- Slide 94: Architecture Decision Tree
- Slide 95: End of Module Assessment
- Slide 96: Module 5 Complete

Module 5: Deep Learning and ML Mastery in Vision and Transfer Learning — Speaker Notes

Total time: ~180 minutes (3 hours) Audience: working professionals; instructors must scaffold for both novices (first deep network in M4) and practitioners (already train models at work).

This module is the most technically dense in the programme. Every slide is labelled FOUNDATIONS, THEORY, or ADVANCED. Guide the room with those labels: green for everyone, blue for stretch, purple for bonus. All exercises are implemented in PyTorch (rewritten this session) with Kailash engines layered on top — ModelVisualizer for diagnostics, OnnxBridge for export, InferenceServer for serving, RLTrainer for RL loops.

Slide 1: Module 5 — Title

Time: ~2 min **Talking points**:

- Welcome the room. Read the provocation aloud: "Every major DL architecture. One paradigm per lesson. All implemented."
- This is the architecture module. 8 lessons, 8 paradigms, all coded end-to-end by the student. Nothing is left as "you will see this later."
- Ask the room: "How many of you trained a neural network in Module 4?" Gauge the mix. You will calibrate the FOUNDATIONS vs ADVANCED balance based on the answer.
- "If beginners look confused": "Module 4 gave you the toolkit — forward pass, backprop, optimisers. Today we use that toolkit to build every major architecture in modern deep learning."
- "If experts look bored": "Even if you have used these architectures at work, very few people have implemented all of them from scratch in a single day. The progression is what matters — you will see how every paradigm shares the same foundation."

Transition: "Here is exactly what you will be able to do by the end of today."

Slide 2: What You Will Learn

Time: ~2 min **Talking points**:

- Walk down the left column — these are concrete capabilities, not abstract topics. Autoencoders, CNNs, LSTMs, transformers, GANs, GNNs, transfer learning, RL.
- Introduce the three depth layers on the right. This module uses them aggressively because the audience is mixed. A banker and a PhD sit in the same room; both leave having learned something new.
- Reassure: "If you are new to deep learning, follow the green FOUNDATIONS markers. The blue THEORY slides go into derivations — welcome, but not required. The purple ADVANCED slides are bonus literature pointers."
- "If beginners look confused": "Nobody is expected to follow every slide. If we hit a blue slide that feels too dense, just note the name and move on — the exercises only assume FOUNDATIONS."
- "If experts look bored": "Stay for the derivations: VAE ELBO, scaled dot-product scaling, Bellman equations. These are the slides that get asked about in interviews."

Transition: "Eight lessons, one paradigm per lesson. Here is the journey."

Slide 3: Your Journey — 8 Lessons

Time: ~1 min **Talking points**:

- Walk through the table quickly — do not read every row. Highlight the progression. Autoencoders bridge from M4. CNNs add spatial. RNNs add temporal. Transformers replace both with attention. GANs generate. GNNs handle graphs. Transfer learning applies everything. RL learns from interaction.
- Key framing: "Each lesson is 15-25 minutes of theory plus an exercise. The exercises build on each other — the conv layers you use in 5.1 become the backbone of 5.2."
- "If beginners look confused": "Think of each lesson as one new vocabulary word for deep learning. By the end of the day, you will speak all eight."

Transition: "You will use four Kailash engines today."

Slide 4: Kailash Engines in Module 5

Time: ~2 min **Talking points**:

- Introduce each engine briefly. OnnxBridge exports any trained model to ONNX format for deployment. InferenceServer serves predictions with warm_cache for production. ModelVisualizer creates training curves, latent space plots, attention maps. RLTrainer wraps DQN, PPO, SAC training loops.
- Emphasise the teaching pattern: "You learn the theory first, build the model in PyTorch, then see how the Kailash engine automates or extends it. That way, when the engine gives you an unexpected result, you know how to debug it."
- "If beginners look confused": "You can think of these as power tools. We will show you the hand-tool version first so you understand what the power tool is doing underneath."
- "If experts look bored": "OnnxBridge and InferenceServer are the production story. Even if you know ONNX, the server abstraction — predict, predict_batch, warm_cache, PredictionResult — saves a lot of boilerplate for serving."

Transition: "Before we start building, let us make sure everyone has the M4 toolkit fresh."

Slide 5: DL Toolkit Refresher (from M4)

Time: ~3 min **Talking points**:

- Recap the M4 toolkit: forward pass, backpropagation, gradient descent (SGD, Adam), dropout, batch normalisation. Everything we build today sits on top of these five ideas.
- Do the quick exercise: "Build a 2-layer classifier in your head. Input -> Linear -> ReLU -> Linear -> Softmax. Loss is cross-entropy. Optimiser is Adam. If that feels familiar, you are ready."
- Name the shift explicitly: M4 trained networks to predict labels (supervised). M5 trains networks to learn representations — compressed (autoencoders), spatial (CNNs), temporal (RNNs), attended (transformers), generated (GANs), transferred (transfer learning), interactive (RL).
- "If beginners look confused": "If any of those words from M4 feel shaky, flag it now. We can spend two minutes on backprop before Lesson 5.1 — it will save you 30 minutes of confusion later."
- "If experts look bored": "The shift from supervised labels to learned representations is what modern deep learning is really about. Every architecture today is a different answer to the question: 'What structure should the latent space have?'"

[PAUSE FOR QUESTIONS — 2 min]

Transition: "Our first architecture is the gentlest — the autoencoder."

Slide 6: Lesson 5.1 — Autoencoders (Title)

Time: ~1 min **Talking points**:

- Title the lesson. Read the provocation: "What if the network had to describe itself in fewer words than it has?"
- Frame the metaphor: compress then reconstruct. If the network can rebuild the input from a compressed version, it has learned the essential structure.
- "If beginners look confused": "Think of the game where you describe a movie in three words and a friend has to guess the movie. If they guess right, your three words captured the essential plot."

Transition: "Three components, one idea — encode, bottleneck, decode."

Slide 7: The Autoencoder Architecture

Time: ~3 min **Talking points**:

- Walk through the three components: encoder compresses, latent space is the bottleneck, decoder reconstructs. The bottleneck is what makes the architecture useful — if the latent dimension were as large as the input, the network would just copy.
- Hammer home "auto": the target IS the input. No labels needed. This is unsupervised learning.
- Use the book-summary analogy: summarise a book in one sentence (encoder). Someone else rewrites the book from your sentence (decoder). The better the summary, the closer the rewrite.
- "If beginners look confused": "Every autoencoder is just these three boxes. Everything we do today — denoising, VAE, convolutional — changes ONE of these three boxes."
- "If experts look bored": "The auto in autoencoder is historically important — it distinguished self-supervised reconstruction from supervised training, long before 'self-supervised learning' became a buzzword."

Transition: "So what does the loss function look like?"

Slide 8: Reconstruction Loss

Time: ~2 min **Talking points**:

- Show the equation: L_recon = || x - decoder(encoder(x)) ||^2. Mean squared error between input and reconstruction.
- Explain what this measures: pixel-by-pixel difference for images, feature-by-feature difference for tabular data. Lower loss = better reconstruction = better features.
- Mention alternatives: binary cross-entropy for normalised inputs (common on MNIST), perceptual loss (compare in feature space), SSIM for structural similarity.
- "If beginners look confused": "MSE just measures how far apart two things are, averaged over every dimension. It is the same idea as measuring the distance between two points, but in a much higher dimensional space."
- "If experts look bored": "The choice of reconstruction loss is an implicit prior on the generative distribution. MSE assumes Gaussian noise on pixels. BCE assumes Bernoulli. That is why the loss affects what the autoencoder prioritises."

Transition: "The simplest autoencoder is fully connected."

Slide 9: Variant 1 — Vanilla Autoencoder

Time: ~3 min **Talking points**:

- Walk through the code block. Encoder: Linear(input_dim, 128) -> ReLU -> Linear(128, latent_dim). Decoder mirrors it. Sigmoid on the output because MNIST pixels are in [0,1].
- Key insight box: if latent_dim equals input_dim, the network just copies. The bottleneck is what forces learning.
- When to use: dimensionality reduction (nonlinear PCA), feature extraction for downstream tasks, anomaly detection (high reconstruction error = anomaly).
- "If beginners look confused": "In the exercise you will feed MNIST digits in with latent_dim=32. The network has to squeeze 784 pixels down to 32 numbers and then rebuild. That forces it to learn what makes a digit a digit."
- "If experts look bored": "Vanilla AE is a nonlinear PCA. If you used a single linear layer with no activation, you would exactly recover the PCA solution."

Transition: "Now what if we corrupt the input to force robust features?"

Slide 10: Variant 2 — Denoising Autoencoder (DAE)

Time: ~3 min **Talking points**:

- Walk through the three steps: corrupt the input with noise, train to reconstruct the clean version, force learning of robust features.
- Show the equation: L_DAE = || x - decoder(encoder(x_tilde)) ||^2 where x_tilde = x + epsilon. The target is the ORIGINAL clean input, not the noisy version.
- Cover the noise types: Gaussian, masking (randomly zero inputs), salt-and-pepper.
- "If beginners look confused": "Imagine a student who can only practise with blurry photocopies of textbook diagrams. They cannot memorise the exact pixels — they have to learn what the diagram MEANS. That is why denoising works."
- "If experts look bored": "DAE is a form of regularisation and early self-supervised learning. Masking noise in particular is the direct ancestor of BERT's masked language modelling, which we see in Lesson 5.4."

Transition: "Now the architectural jump — what if the latent space were a probability distribution?"

Slide 11: Variant 3 — Variational Autoencoder (VAE)

Time: ~3 min **Talking points**:

- The key innovation: the encoder outputs mu and sigma (mean and standard deviation), not a fixed point. You SAMPLE the latent code from a Gaussian parameterised by those outputs.
- Why it matters: you can GENERATE new data by sampling from the latent space. Vanilla AE can only reconstruct; VAE can create.
- Use the coordinate analogy: "Vanilla AE says this digit lives at (3.2, -1.7). VAE says this digit lives SOMEWHERE AROUND (3.2, -1.7) with some uncertainty. That fuzziness means nearby points are also valid digits — which is what lets you generate."
- Formally, the encoder approximates the posterior q_phi(z|x) approximates p(z|x) and the decoder models the likelihood p_theta(x|z).
- "If beginners look confused": "The practical punchline is: VAE is the first model we build that can make NEW MNIST digits. You will literally generate a new handwritten 7 from pure noise in the exercise."
- "If experts look bored": "This is amortised variational inference. The encoder is the inference network — we are learning an approximation to the intractable true posterior, and the ELBO is the training signal."

Transition: "The VAE loss is called the ELBO. Here it is."

Slide 12: VAE Loss — The ELBO

Time: ~4 min **Talking points**:

- Write the ELBO carefully: L_VAE = E_q[log p(x|z)] - D_KL(q_phi(z|x) || p(z)). Two terms. Maximise this (or minimise its negative in practice).
- Term 1 is the reconstruction: how well does the decoder rebuild the input from a sample z. Familiar from vanilla AE.
- Term 2 is the KL divergence: how close is the learned latent distribution to the prior N(0, I). This acts as a regulariser. Without it, the VAE collapses back to a vanilla AE with isolated latent points and no generative capacity.
- Advanced box: the KL for Gaussians has a closed form. No sampling needed for that term. Write it out: D_KL = -(1/2) Sum (1 + log sigma^2 - mu^2 - sigma^2).
- "If beginners look confused": "Term 1 says 'reconstruct well.' Term 2 says 'keep the latent space tidy.' You need both — reconstruction alone gives you a vanilla AE; tidiness alone gives you random noise."
- "If experts look bored": "ELBO = Evidence Lower BOund. It is a lower bound on log p(x), the true data likelihood. Maximising ELBO maximises a lower bound on the log-likelihood — we cannot compute the true likelihood because of the intractable posterior."

Transition: "But we just said the latent code is sampled. How does gradient flow through a random sample?"

Slide 13: The Reparameterisation Trick

Time: ~4 min **Talking points**:

- State the problem plainly: sampling z ~ N(mu, sigma^2) is stochastic. Gradients cannot flow through random sampling. Backprop breaks.
- Solution: z = mu + sigma * epsilon, where epsilon ~ N(0, I). Move the randomness out to an INPUT. Now gradients flow through mu and sigma because they are deterministic paths.
- Walk through the three practical steps: encoder outputs mu and log(sigma^2) (log-variance for numerical stability), sample epsilon from a standard normal, compute z = mu + exp(0.5 * log sigma^2) * epsilon.
- Ask the room: "Why log-variance instead of variance directly?" Answer: numerical stability and ensuring sigma is always positive via the exponential.
- "If beginners look confused": "The trick is moving the coin flip from inside the network to outside. Now the network only has to decide WHERE to sample (mu) and HOW WIDE (sigma). The actual coin flip is a fixed input, not a trainable operation."
- "If experts look bored": "This is the single most elegant idea in the VAE paper. The reparameterisation gradient has lower variance than the score-function gradient (REINFORCE), which is why VAEs train so much better than early stochastic networks."

Transition: "Now let us swap fully connected layers for convolutions."

Slide 14: Variant 4 — Convolutional Autoencoder

Time: ~3 min **Talking points**:

- Motivation: fully connected layers ignore spatial structure. Conv layers preserve spatial relationships. Far fewer parameters for image data.
- Architecture: encoder uses Conv2d + ReLU + MaxPool to downsample. Decoder uses ConvTranspose2d + ReLU to upsample. Mirror structure.
- Walk through the dimensions: 28x28x1 -> 14x14x16 -> 7x7x4 in the encoder. Decoder reverses.
- "If beginners look confused": "The conv layer is just a sliding filter — we see exactly how it works in the very next lesson. For now, trust that it keeps spatial structure."
- "If experts look bored": "ConvTranspose has its own artefacts (checkerboard patterns) that modern architectures address with nearest-neighbour upsampling + regular conv."

Transition: "There are many more variants — here is a quick survey."

Slide 15: Additional Autoencoder Variants (Survey)

Time: ~2 min **Talking points**:

- Quick walk through the table: Sparse AE (L1 penalty on activations), Contractive AE (Jacobian penalty), Stacked AE (layerwise pretraining), Recurrent AE (LSTM encoder-decoder), CVAE (conditional VAE).
- Advanced box: VQ-VAE (Vector Quantised) discretises the latent space. Used in DALL-E 1. Bridges autoencoders and modern generative AI.
- "Students do not implement these — the four variants you build in the exercise cover the core ideas."
- "If experts look bored": "VQ-VAE is worth knowing for anyone interested in modern generative models. The discrete latent space is what lets you use a transformer as the prior — which is exactly how DALL-E 1 and VQ-GAN work."

Transition: "Time to build. Here is the exercise."

Slide 16: Exercise 5.1 — Autoencoder Workshop

Time: ~1 min **Talking points**:

- Five tasks: vanilla AE on MNIST (latent_dim=32), denoising AE with noise comparison, VAE with reparameterisation, VAE generation from pure noise, convolutional AE.
- Assessment: all four variants trained, VAE produces plausible new images, latent space visualised (t-SNE or PCA), reconstruction quality compared.
- Stretch goal: interpolate between two digits in VAE latent space. Watch the smooth transition — 7 morphing into 1 through the latent space is the most fun moment of the morning.
- Note: exercise uses PyTorch directly. ModelVisualizer provides the latent space plots once you hand it the trained model.

Transition: "Quick summary before we move to CNNs."

Slide 17: Lesson 5.1 Summary

Time: ~1 min **Talking points**:

- One-line takeaway: autoencoders learn compressed representations by reconstructing their own input through a bottleneck.
- Three key equations to remember: reconstruction loss, ELBO, reparameterisation.

- Bridge: the convolutional autoencoder encoder is essentially a CNN feature extractor. In Lesson 5.2 we replace the decoder with a classifier head — and we have a CNN.

Transition: "The conv layers we just used have a whole architecture of their own. Let us go deeper."

Slide 18: Lesson 5.2 — CNNs and Computer Vision (Title)

Time: ~1 min **Talking points**:

- Title the lesson. Read the provocation: "What if the model could see edges, textures, and objects — the way you do?"
- Frame it: CNNs are the workhorse of computer vision. You already used conv layers in the autoencoder; now we build a full classification pipeline and look at how the architecture evolved from 1998 to today.
- "If experts look bored": "We cover LeNet to ResNet to ViT in one lesson. The arc is short but every step matters."

Transition: "Start with the building block — the convolution."

Slide 19: The Convolution Operation

Time: ~3 min **Talking points**:

- Walk through filter (kernel), stride, padding, feature map. A filter is a small matrix (e.g. 3x3) that slides over the input. Stride is how far it moves each step. Padding adds zeros around the border.
- Output size formula: (W - F + 2P) / S + 1. Essential for architecture design.
- Worked example: 28x28 input, 3x3 filter, stride 2, no padding -> (28 - 3 + 0) / 2 + 1 = 13. Ask students to compute a few more on scratch paper.
- "If beginners look confused": "Think of a magnifying glass scanning a photo, looking for a specific pattern. At each position it outputs a number — how strongly the pattern matches there. The feature map is that grid of numbers."
- "If experts look bored": "The output size formula is a constraint you hit immediately when designing networks. Every architecture paper starts with 'we use 3x3 convs with stride 1 and padding 1 to preserve spatial size.' Now you know why."

Transition: "After conv comes pooling — how we build hierarchy."

Slide 20: Pooling and Feature Hierarchy

Time: ~2 min **Talking points**:

- Pooling reduces spatial dimensions: max pooling (dominant in practice), average pooling, global average pooling (GAP).
- Feature hierarchy insight: early layers learn low-level features (edges, textures), deep layers learn high-level features (objects, faces). This is the empirical reason transfer learning works.
- GAP over FC: increasingly preferred at the end of networks. Fewer parameters, less overfitting.
- "If beginners look confused": "Max pooling just takes the brightest pixel in each 2x2 block. It makes the feature map smaller and keeps the strongest signal."
- "If experts look bored": "GAP replacing FC layers is the architectural trick that made transfer learning robust — you can attach a single linear classifier head to any pre-trained GAP output."

Transition: "Now let us trace how CNNs evolved."

Slide 21: CNN Architecture Evolution

Time: ~3 min **Talking points**:

- Walk the timeline: LeNet-5 (1998, handwritten digits, 7 layers), AlexNet (2012, deeper, ReLU, dropout, the ImageNet breakthrough), VGGNet (2014, very small 3x3 filters, depth matters), GoogLeNet/Inception (2015, multiple filter sizes in parallel).
- The trend is clear: deeper networks learn better features, but training them is hard. Each architecture solved the training problem of the previous generation.
- "If beginners look confused": "For most of the last decade, the recipe was 'take the best architecture, train it bigger on more data.' The architectures we talk about now are the ones that survived."
- "If experts look bored": "The Inception module foreshadows many later ideas — parallel pathways, channel mixing. It is the architectural ancestor of modern multi-branch designs."

Transition: "And then came ResNet — the single most important architectural innovation in modern deep learning."

Slide 22: ResNet — Skip Connections

Time: ~4 min **Talking points**:

- The equation: H(x) = F(x) + x. The network learns a residual F(x), and the identity x is added back. Skip connection.
- Why it matters: walk through the gradient. Even if F(x) has vanishing gradients, the +1 from the identity ensures signal flows. This is why ResNets can be 152 layers deep where VGG fails beyond 20.
- Architectural reframing: "Instead of learning to transform x into something new, the network learns what to ADD to x. If nothing needs to change, the network outputs zero — and that is much easier to learn than an identity function from scratch."
- "If beginners look confused": "Think of it like editing a Wikipedia article. It is much easier to suggest 'add this sentence' than to rewrite the whole article. ResNet's skip connection lets each layer suggest small edits."
- "If experts look bored": "Residual connections also make the loss landscape smoother — there is a famous paper visualising the loss surface of ResNet vs VGG. The ResNet surface is convex-looking; VGG's is chaotic."

Transition: "Modern CNNs add a few more tricks on top. First — SE blocks."

Slide 23: Modern Enhancement — SE Blocks

Time: ~3 min **Talking points**:

- Squeeze-and-Excitation block: s = sigmoid(W2 * ReLU(W1 * GAP(x))). Global average pool, then two FC layers, then sigmoid, then rescale channels.
- Intuition: not all feature channels are equally important for every input. SE learns to boost relevant channels and suppress irrelevant ones. Per-channel gating.
- Minimal parameter overhead, measurable accuracy gains. Can be added to any CNN architecture — ResNet, VGG, MobileNet.

- "If beginners look confused": "Imagine the conv layer produces 64 channels. SE looks at all 64 and says 'channel 17 is very relevant for this input — turn it up. Channel 42 is noise — turn it down.' It is a learned equaliser."
- "If experts look bored": "SE won the ImageNet 2017 classification challenge as SENet. It is the template for all later attention-over-channels designs, including many vision transformer variants."

Transition: "There are a few more training tricks that separate research-grade from production-grade CNN training."

Slide 24: Modern Training Enhancements

Time: ~3 min **Talking points**:

- Kaiming initialisation: proper for ReLU networks. Fixes the variance blow-up that plagued early deep networks.
- Mixed precision training: FP16 for most operations, FP32 for accumulation. Roughly free 2x speedup on modern GPUs.
- Mixup augmentation: combine two images linearly, combine their labels linearly. Produces smoother decision boundaries.
- Label smoothing: replace one-hot labels with (1 - epsilon) for the target class and epsilon/(K-1) elsewhere. Prevents overconfident predictions.
- Gradient flow analysis: monitor the gradient norms at each layer during training. Signals when vanishing or exploding.
- "If beginners look confused": "Use all of these in the exercise. They each give a small accuracy bump and compound together."
- "If experts look bored": "Mixup is a form of vicinal risk minimisation — training on the vicinity of each data point. It is interpretable as Bayesian data augmentation."

Transition: "One more preview — Vision Transformers. They matter but we cover them fully in Lesson 5.4."

Slide 25: Vision Transformers (ViT) — Brief Introduction

Time: ~2 min **Talking points**:

- Key idea: split an image into patches (e.g. 16x16 squares), treat each patch as a token, feed the sequence of tokens through a transformer encoder. Classification head on top.
- This is 2020's breakthrough paper "An Image is Worth 16x16 Words." ViT matched CNN performance on ImageNet at scale.
- Brief intro only — full attention derivation comes in Lesson 5.4. The seed is planted: transformers are not just for text.
- "If beginners look confused": "Do not worry about attention yet. The only thing to remember is that images can be chopped into patches and fed through the same architecture we will build for text. That is the magic."
- "If experts look bored": "ViT scales better than CNNs above a certain dataset size threshold (around 300M images). Below that, CNNs with strong inductive biases still win. That is why mixed architectures like Swin are common in practice."

Transition: "Now let us meet the first Kailash engine of the day — OnnxBridge."

Slide 26: Kailash Bridge — OnnxBridge

Time: ~2 min **Talking points**:

- OnnxBridge wraps the PyTorch ONNX exporter with validation and metadata. Students export their trained CNN at the end of the exercise. This is the first step toward production deployment, completed fully in Lesson 5.7.
- Why ONNX: it is the cross-platform neural network format. One file loads in Python, C++, JavaScript, mobile, and edge runtimes. You train once, deploy anywhere.
- Pattern: train in PyTorch, export with OnnxBridge, serve with InferenceServer (Lesson 5.7).
- "If beginners look confused": "You do not need to understand ONNX internals. The engine takes your trained model and writes a file. Someone else on your team can load that file on a different device and run predictions."
- "If experts look bored": "OnnxBridge validates the exported graph against the original PyTorch model on sample inputs. This catches the classic export bugs — dynamic shapes, unsupported operators, mismatched dtypes."

Transition: "Time to build. Here is the exercise."

Slide 27: Exercise 5.2 — CNN Classification Pipeline

Time: ~1 min **Talking points**:

- Progressive build: start with a simple CNN, add a ResBlock, add an SE block, apply modern training enhancements. You should see clear accuracy jumps at each stage.
- Dataset: Fashion-MNIST or mask detection. Both are in the data loader.
- Final step: export to ONNX with OnnxBridge. This connects to the deployment story in Lesson 5.7.
- Note: exercise uses PyTorch. Kailash layers on top for visualisation and export.

Transition: "Quick reference for the key formulas."

Slide 28: Lesson 5.2 Key Formulas

Time: ~1 min **Talking points**:

- Four formulas to carry forward: conv output size (W - F + 2P) / S + 1, ResNet skip H(x) = F(x) + x, SE gating s = sigmoid(W2 ReLU(W1 GAP(x))), Kaiming init variance 2/n_in for ReLU.
- "Students should be able to compute conv output sizes by hand and explain in one sentence why the +x in ResNet prevents vanishing gradients."

Transition: "Lesson summary."

Slide 29: Lesson 5.2 Summary

Time: ~1 min **Talking points**:

- CNNs capture spatial features through convolutions, pooling, and hierarchical depth. ResNet skip connections enable deep networks to train. SE blocks add channel recalibration. Modern training tricks compound for real accuracy gains.

- Bridge: CNNs handle spatial data (images, grids). What about sequential data (text, time series)? We need an architecture that remembers previous inputs. That is the RNN.

Transition: "From space to time."

Slide 30: Lesson 5.3 — RNNs and Sequence Models (Title)

Time: ~1 min **Talking points**:

- Title the lesson. The transition from spatial to temporal deserves a pause.
- The key difference: in an image, pixel (3,4) is always next to pixel (3,5). In a sequence, the meaning of a word depends on what came before. Order matters. History matters.
- "If experts look bored": "RNNs feel dated in 2026, but they are still the clearest way to teach sequence modelling and they still ship in production — especially LSTMs on edge devices where transformers are too heavy."

Transition: "What makes an RNN recurrent?"

Slide 31: Recurrent Neural Networks

Time: ~3 min **Talking points**:

- Draw the unrolled RNN on the whiteboard. The same weight matrix W is applied at every time step. The hidden state h_t carries memory from previous steps.
- Equation: h_t = tanh(W_x x_t + W_h h_{t-1} + b).
- Introduce the central problem: vanishing gradients. When you unroll the RNN over many time steps, gradients either shrink to zero or explode. Long-range dependencies are unreachable.
- This is why vanilla RNNs forget: by time step 20, the influence of time step 1 has been multiplied by W_h twenty times and effectively erased.
- "If beginners look confused": "Imagine whispering a message down a line of 20 people. By the end, the message is unrecognisable. The vanilla RNN has this exact problem."
- "If experts look bored": "The vanishing gradient analysis goes back to Bengio 1994. It motivated every subsequent architecture in this lesson."

Transition: "LSTM is the fix."

Slide 32: LSTM — Long Short-Term Memory

Time: ~3 min **Talking points**:

- The LSTM has four components: cell state (the memory highway), forget gate, input gate, output gate.
- Key architectural insight: the cell state C_t is updated by ADDITION, not multiplication. Gradients flow through the addition unchanged. Even if f_t is near 1, the gradient passes through.
- This is the cell state highway — long-range gradients travel along C_t without being crushed by repeated multiplication.
- "If beginners look confused": "You do not have to memorise all the equations. What matters is the idea: the LSTM has a separate memory pathway that gradients can travel down without being squashed."
- "If experts look bored": "LSTM is a decade ahead of its time — Hochreiter and Schmidhuber 1997. The cell-state pathway is structurally identical to residual connections, which the computer vision community

rediscovered as ResNet in 2015."

Transition: "Let us derive all five gate equations."

Slide 33: LSTM — All Five Equations

Time: ~4 min **Talking points**:

- Walk through each equation slowly. Concatenation [h_{t-1}, x_t] means the gate decision depends on BOTH the previous hidden state and the current input.
- Forget gate: f_t = sigma(W_f [h_{t-1}, x_t] + b_f). Decides what to throw away from the cell state.
- Input gate: i_t = sigma(W_i [h_{t-1}, x_t] + b_i). Decides what new information to store.
- Candidate: C_tilde = tanh(W_C [h_{t-1}, x_t] + b_C). New candidate values.
- Cell update: C_t = f_t * C_{t-1} + i_t * C_tilde. Forget some old, add some new.
- All gates use sigmoid (output in [0,1]) because they are soft binary decisions. The candidate uses tanh (output in [-1,1]) because it represents new information.
- "If beginners look confused": "Forget some of the past. Decide what new thing to remember. Add them together. That is the cell update."
- "If experts look bored": "The sigmoid/tanh pairing is empirical. GRU simplifies by merging gates; QRNN replaces tanh with parallel convolutions. The LSTM design has survived mostly untouched for 28 years."

Transition: "One more equation — how the hidden state comes out."

Slide 34: LSTM — Output Gate and Hidden State

Time: ~3 min **Talking points**:

- Output gate: o_t = sigma(W_o [h_{t-1}, x_t] + b_o). Decides which parts of the cell state to expose.
- Hidden state: h_t = o_t * tanh(C_t). The cell state is squashed through tanh, then masked by the output gate.
- h_t goes to the next time step AND is used for prediction at the current time step. C_t is internal memory only — it never leaves the cell.
- "If beginners look confused": "Think of C_t as private notes and h_t as what you say out loud. The output gate decides which parts of your notes to share."
- "If experts look bored": "The tanh(C_t) squash is what keeps the hidden state bounded. Without it, the cell state could grow unboundedly over long sequences, causing numerical issues in downstream layers."

Transition: "GRU is the simpler cousin of LSTM."

Slide 35: GRU — Gated Recurrent Unit

Time: ~3 min **Talking points**:

- GRU merges the forget and input gates into a single update gate z_t. Adds a reset gate r_t that controls how much of the previous hidden state to use when computing the new candidate.
- Fewer parameters, faster training, similar performance on most tasks.
- In practice, try both on your task and pick whichever trains better.
- "If beginners look confused": "GRU and LSTM do the same job. GRU has fewer knobs. For most problems it is the practical default."

- "If experts look bored": "Empirically the gap between GRU and LSTM is within noise for most benchmarks. The cases where LSTM wins are extremely long sequences and certain reinforcement learning settings."

Transition: "Now the idea that leads us to transformers — attention."

Slide 36: Attention Mechanisms for Sequences

Time: ~3 min **Talking points**:

- The bottleneck problem with RNNs: the entire sequence context must fit in the final hidden state. For long sequences, information is lost.
- Attention's answer: instead of forcing everything through the last hidden state, let the model weight ALL hidden states by relevance and combine them.
- Temporal attention: for each prediction, compute attention weights over all time steps. Weighted sum of hidden states becomes the context.
- This is the seed of self-attention in Lesson 5.4.
- "If beginners look confused": "Instead of making the student write a 500-word essay from memory, give them permission to look back at the source text and highlight the sentences they need. That is attention."
- "If experts look bored": "The Bahdanau attention paper (2014) introduced additive attention for neural machine translation. The shift from encoder-decoder with a bottleneck to encoder-decoder with attention was the prelude to 'Attention Is All You Need' three years later."

Transition: "How do we measure sequence models?"

Slide 37: Sequence Model Metrics

Time: ~2 min **Talking points**:

- Perplexity: PP = exp(-1/N Sum log P(w_i)). Standard metric for language models. Lower is better. Intuitively, "how surprised is the model on average per token."
- BLEU: n-gram overlap between machine and human translations. Mentioned for completeness.
- Sequence accuracy: did the whole predicted sequence match? Strict metric.
- Cross-entropy loss: the training objective itself. Related to perplexity via PP = exp(cross entropy).
- For the exercise, students use MAE/RMSE on time series prediction and cross-entropy on text generation. BLEU is not assessed in this module.
- "If experts look bored": "Perplexity is the exponential of cross-entropy, so a perplexity of 100 means the model is as uncertain as if it were choosing uniformly among 100 options. A good English language model today is around 10-20."

Transition: "Two practical applications bring RNNs to life."

Slide 38: Applications — Finance and Text

Time: ~2 min **Talking points**:

- Financial time series: LSTM on Singapore stock prices. Use technical indicators (RSI, MACD, Bollinger Bands) as features. Real professional context.

- Text generation: character-level LSTM trained on Shakespeare. Generate new Shakespearean-sounding text. Fun and pedagogically effective — students see a working generative model before we reach GANs.
- Both are from-scratch PyTorch implementations using nn.LSTM and nn.GRU.
- "If beginners look confused": "Stock prices are just a sequence of numbers over time. Shakespeare is a sequence of characters. Same architecture, different data."
- "If experts look bored": "The Shakespeare exercise is Karpathy's classic char-RNN from 2015. Still a beautifully instructive baseline."

Transition: "Time for the sequence exercise."

Slide 39: Exercise 5.3 — Sequence Modelling

Time: ~1 min **Talking points**:

- Three tasks: LSTM for stock prediction with technical indicators, GRU vs LSTM comparison, character-level LSTM for Shakespeare generation. Bonus: add a temporal attention layer.
- Critical: gradient clipping is non-negotiable for RNN training. clip_grad_norm_ to 1.0 or 5.0. Without it, exploding gradients crash training on the first long sequence.
- Students should include gradient clipping in every RNN training loop they ever write.

Transition: "Quick summary before transformers."

Slide 40: Lesson 5.3 Summary

Time: ~1 min **Talking points**:

- RNNs process sequences one step at a time, carrying a hidden state. Vanilla RNNs forget quickly due to vanishing gradients. LSTMs and GRUs add gated cell states that preserve long-range information. Attention lets the model look back across all time steps.
- Bridge: the attention mechanism is the seed. What if attention was ALL you needed? That is the question the transformer paper answered.

Transition: "The paper that changed everything."

Slide 41: Lesson 5.4 — Transformers (Title)

Time: ~1 min **Talking points**:

- Title the lesson. This is "Attention Is All You Need," Vaswani et al. 2017. The paper that made GPT, BERT, Claude, and every modern LLM possible.
- The lesson derives self-attention from scratch. By the end, students will have implemented it in PyTorch without looking at a reference.
- "If beginners look confused": "Every AI product you have heard about in the last three years is built on this architecture. What we cover in the next 25 minutes is the foundation of the industry."

Transition: "Start with the core idea — query, key, value."

Slide 42: Self-Attention — The Core Idea

Time: ~3 min **Talking points**:

- Every input token is projected into three vectors: Query (Q), Key (K), Value (V). These are learned linear projections of the same input.
- The Q of one token asks a question. The K of every other token advertises what it contains. The dot product Q . K measures similarity. Softmax turns similarities into weights. The weighted sum of values is the output.
- Use the library analogy: "If you are looking for information about cats, your query is 'cats.' Each book on the shelf has a key — its title. You compare your query against every key. High similarity = high attention weight. The weighted sum of the books' contents (values) is what you read."
- "If beginners look confused": "Three copies of the same input, each with a different purpose. Query says what you want. Key says what I have. Value says what I contain. That is the whole trick."
- "If experts look bored": "QKV is also the generalisation of content-based memory addressing. Neural Turing Machines used something similar before transformers formalised it."

Transition: "Now the equation."

Slide 43: Scaled Dot-Product Attention

Time: ~4 min **Talking points**:

- Write the equation: Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V.
- Walk through each step: (1) compute Q K^T — a matrix of raw similarity scores between every query and every key, (2) divide by sqrt(d_k) — the scaling, (3) softmax over each row — turn scores into weights, (4) multiply by V — weighted sum of values.
- The scaling is crucial and often asked in interviews. Without it, for large d_k the dot products become large, the softmax saturates (nearly one-hot), and gradients vanish.
- For d_k = 512, the expected magnitude of dot products is around sqrt(512) approximately 22.6. Without scaling, softmax would produce essentially a hard argmax. Scaling keeps the distribution soft.
- "If beginners look confused": "The key insight: this one equation is the entire transformer block. Everything else is just stacking and normalising this operation."
- "If experts look bored": "The dot-product form is efficient because it can be computed as a single batched matrix multiplication. Additive attention (from Bahdanau) requires a small MLP at every pair, which is much slower."

Transition: "Why exactly do we divide by sqrt(d_k)?"

Slide 44: Why Divide by sqrt(d_k)?

Time: ~3 min **Talking points**:

- Derivation: assume each component of Q and K has mean 0 and variance 1. The dot product Q . K = sum of d_k such products, so has variance d_k. Standard deviation sqrt(d_k).
- Without scaling, dot products grow with dimension. Softmax of a large value becomes a one-hot vector. Gradients at the non-peak positions are zero. Training stalls.
- Dividing by sqrt(d_k) restores variance to O(1) regardless of dimension. Softmax stays soft. Gradients flow.
- "If beginners look confused": "It is a variance stabilisation trick. Keep the input to softmax in a reasonable range so the gradients do not die."
- "If experts look bored": "This is the same kind of argument behind layer normalisation, Kaiming init, and batch norm — maintain unit variance through the network. It is everywhere once you see it. And yes, the

scaling question appears in actual ML interviews — companies ask this."

Transition: "One head is not enough. Multi-head attention."

Slide 45: Multi-Head Attention

Time: ~3 min **Talking points**:

- Equation: MultiHead(Q, K, V) = Concat(head_1, …, head_h) W^O. Each head is scaled dot-product attention with its own learned projections.
- Intuition: different heads attend to different relationships. One head might learn syntactic dependencies, another semantic similarity, another positional patterns.
- Compute trick: per-head dimension is reduced proportionally (d_model / h), so total compute stays roughly the same as a single head at full dimension.
- Use the analogy: "Instead of one person reading a document, 8 people each read it for a different purpose, then combine their notes."
- "If beginners look confused": "Multi-head attention is just running the attention operation 8 times in parallel with different learned projections, then concatenating the results. Diversity of attention patterns."
- "If experts look bored": "Empirically, different heads specialise — there are famous visualisations showing heads that attend to subject-verb, head of phrase, punctuation boundaries. But many heads are redundant, which is why pruning research exists."

Transition: "One problem — transformers have no idea about order."

Slide 46: Positional Encoding

Time: ~3 min **Talking points**:

- Attention is permutation-invariant. If you shuffle the tokens, the output shuffles the same way. So the transformer has no inherent sense of order.
- Fix: add a position vector to each token embedding. The original paper uses sinusoidal encodings. Each dimension oscillates at a different frequency.
- Equation: PE(pos, 2i) = sin(pos / 10000^(2i/d)), PE(pos, 2i+1) = cos(pos / 10000^(2i/d)).
- Low dimensions change slowly (capture global position). High dimensions change rapidly (capture local position).
- The sinusoidal form has a nice property: the model can learn to attend to RELATIVE positions because PE(pos + k) is a linear function of PE(pos) for any fixed offset k.
- "If beginners look confused": "Add a position label to each token so the model knows what came first. The sine/cosine form is one clean way to do it."
- "If experts look bored": "Learned positional embeddings (used in BERT) also work and are often preferred in practice. Rotary position embeddings (RoPE) are the current SOTA for long-context models."

Transition: "Now the full architecture — encoder and decoder."

Slide 47: Transformer Architecture

Time: ~3 min **Talking points**:

- Encoder block: multi-head self-attention -> residual + layer norm -> feed-forward -> residual + layer norm. Stack N of these.
- Decoder block: masked multi-head self-attention -> residual + layer norm -> cross-attention to encoder -> residual + layer norm -> feed-forward -> residual + layer norm. Stack N of these.
- The decoder has two types of attention: masked self-attention (prevents looking at future tokens during autoregressive generation) and cross-attention (queries are from the decoder, keys and values are from the encoder).
- Residual connections and layer norm are critical for training stability. Removing either causes training to diverge.
- "If beginners look confused": "The encoder understands the input. The decoder generates the output, token by token, looking back at the encoder whenever it needs context. That is translation in one paragraph."
- "If experts look bored": "Modern LLMs like GPT use decoder-only architectures (no encoder, no cross-attention). BERT uses encoder-only. Only translation and sequence-to-sequence tasks still use the full encoder-decoder."

Transition: "The transformer family tree."

Slide 48: Transformer Variants

Time: ~2 min **Talking points**:

- Three branches of the family: encoder-only (BERT, RoBERTa — understanding, classification), decoder-only (GPT, LLaMA — generation), encoder-decoder (T5, BART — translation, summarisation).
- Efficiency variants: Transformer-XL (segment-level recurrence for long contexts), Reformer, Longformer (sparse attention patterns for long sequences).
- ViT: decoder-only applied to image patches.
- Students should know the landscape, not memorise every variant. The distinction that matters is encoder vs decoder vs both.
- BERT is the fine-tuning exercise in this lesson. GPT is the generation paradigm covered in M6.
- "If experts look bored": "The encoder-only vs decoder-only choice maps directly to discriminative vs generative. BERT's masked language modelling is bidirectional; GPT's causal masking is autoregressive. Both are self-supervised pretraining objectives."

Transition: "Consolidation moment — we have seen four paradigms."

Slide 49: Consolidation — Four Paradigms Compared

Time: ~3 min **Talking points**:

- Students have now seen autoencoder, CNN, RNN, and Transformer. Pause here and connect them.
- Quick matching game. Call on the room: "If I give you a dataset of satellite images, which architecture?" (CNN or ViT.) "Patient records over time?" (LSTM or Transformer.) "Unlabelled images for pretraining?" (Autoencoder or self-supervised ViT.) "A social network?" (GNN, coming in 5.6.)
- The table on the slide shows data type -> architecture -> when to use. Keep it on screen while you work through examples.
- "If beginners look confused": "You do not need to choose architectures in the exercise — we tell you which one. But by the end of the day, you should be able to look at a new problem and pick the right family in 30 seconds."
- "If experts look bored": "The architecture choice is increasingly converging — transformers can do all of this. But the inductive biases still matter: CNNs for small vision data, LSTMs for edge devices, MLPs for tabular. No silver bullet."

[PAUSE FOR QUESTIONS — 2 min]

Transition: "Time for the transformer exercise."

Slide 50: Exercise 5.4 — Transformers

Time: ~1 min **Talking points**:

- Three tasks: (1) implement scaled dot-product attention from scratch in PyTorch — no library calls, just torch.matmul and torch.softmax, (2) fine-tune BERT for text classification using HuggingFace (TREC-6 dataset), (3) compare accuracy and training time with an LSTM baseline from Lesson 5.3.
- The from-scratch implementation is crucial. Students should be able to write the attention function without a reference by the end.
- The LSTM comparison quantifies the transformer advantage on this task — usually a few accuracy points and a different training profile.

Transition: "Key formulas reference."

Slide 51: Lesson 5.4 Key Formulas

Time: ~1 min **Talking points**:

- Four equations to know by heart: scaled dot-product attention, multi-head attention, sinusoidal positional encoding, layer norm.
- "These four lines are the core of every modern AI product. If you remember nothing else from today, remember these."

Transition: "Summary and bridge."

Slide 52: Lesson 5.4 Summary

Time: ~1 min **Talking points**:

- Self-attention lets every token attend to every other token. Scaling by sqrt(d_k) stabilises training. Multi-head runs attention in parallel with diverse patterns. Positional encoding restores order. The full transformer stacks these blocks with residuals and layer norm.
- Bridge: we have covered four paradigms for LEARNING from data. Now we shift to GENERATING new data. Enter the GAN.

Transition: "Two networks playing a game."

Slide 53: Lesson 5.5 — GANs and Diffusion (Title)

Time: ~1 min **Talking points**:

- Title the lesson. GANs are one of the most creative ideas in machine learning.
- Set up the core concept: a generator creates fake data, a discriminator tries to tell fake from real. They train against each other. The competition drives both to improve.

- "If beginners look confused": "Imagine a forger and a detective. The forger gets better at faking, the detective gets better at spotting fakes, and they keep training each other up."

Transition: "Meet the two players."

Slide 54: GAN — Generator vs Discriminator

Time: ~3 min **Talking points**:

- Generator G: takes random noise z from a prior distribution, outputs a fake sample G(z) that should look real.
- Discriminator D: takes a sample (real or fake), outputs probability that the sample is real.
- Training is alternating: update D to better distinguish, update G to better fool.
- The loss is binary cross-entropy on the discriminator outputs.
- "If beginners look confused": "Two networks, two losses, alternating updates. That is the whole framework. The tricky part is making the alternation stable."
- "If experts look bored": "The zero-sum framing is beautiful but brittle. In practice, you balance G and D carefully. If D is too strong, G gets zero gradients. If G is too strong, D can only guess randomly and cannot provide signal."

Transition: "The mathematical objective."

Slide 55: GAN Minimax Objective

Time: ~3 min **Talking points**:

- The objective: min_G max_D [E_x[log D(x)] + E_z[log(1 - D(G(z)))]]. Discriminator maximises this (distinguish well), generator minimises it (fool D).
- The inner max is solved at D*(x) = p_data(x) / (p_data(x) + p_G(x)). Substituting back gives a Jensen-Shannon divergence between real and generated distributions.
- Practical training trick: instead of minimising log(1 - D(G(z))), maximise log D(G(z)). The original objective has vanishing gradients when G is poor (D(G(z)) close to 0 means log(1 - 0) close to 0, flat gradient). The trick gives sharp gradients early in training.
- "If beginners look confused": "The formula looks scary but the idea is simple: two players playing a game. The math just makes the game rules precise."
- "If experts look bored": "This is the non-saturating loss from Goodfellow's original paper. Many modern GAN papers skip the derivation and just use it. The JS divergence interpretation also motivates WGAN — JS is discontinuous when distributions do not overlap, which is why vanilla GANs are unstable."

Transition: "Let us look at the architectural standard — DCGAN."

Slide 56: DCGAN — Deep Convolutional GAN

Time: ~3 min **Talking points**:

- DCGAN established the standard architecture guidelines for convolutional GANs: no max pooling (use strided convolutions instead), batch normalisation in both G and D, ReLU in G except the last layer (tanh), LeakyReLU in D.

- Generator: starts from a noise vector, uses ConvTranspose2d to upsample to an image. Each layer doubles spatial dimension while halving channels.
- Discriminator: standard CNN that outputs a single probability.
- These guidelines are empirical but effective. This is what students implement in the exercise.
- "If beginners look confused": "DCGAN is just 'take a CNN and turn it into a generator plus a discriminator with these specific rules.' The rules came from trying many things and writing down what worked."
- "If experts look bored": "DCGAN was 2015. The guidelines have held up surprisingly well — modern GAN architectures still use most of them. Progressive growing (StyleGAN) and attention (SAGAN) are the main refinements since."

Transition: "WGAN is the stability upgrade."

Slide 57: WGAN — Wasserstein GAN

Time: ~3 min **Talking points**:

- Key idea: replace Jensen-Shannon divergence with Wasserstein distance (Earth Mover's Distance). Gradients are smooth everywhere — even when real and generated distributions do not overlap.
- Equation: min_G max_D [E[D(x)] - E[D(G(z))]] with a Lipschitz constraint on D.
- Original WGAN enforced the constraint with weight clipping. Modern WGAN-GP uses a gradient penalty: add lambda * (||grad D||_2 - 1)^2 to the discriminator loss.
- Practical benefits: fewer mode collapse issues, the discriminator loss correlates with image quality, training is much more stable.
- "If beginners look confused": "WGAN fixes the training instability of vanilla GAN by using a better distance between distributions. You get cleaner training curves and less 'did the model collapse?' debugging."
- "If experts look bored": "The Lipschitz constraint is what makes Wasserstein distance computable via the Kantorovich-Rubinstein duality. Gradient penalty is an elegant soft enforcement — it penalises the discriminator when its gradient norm strays from 1."

Transition: "The GAN family in one table."

Slide 58: GAN Variants (Survey)

Time: ~2 min **Talking points**:

- Survey table: Conditional GAN (cGAN — class-conditioned generation), CycleGAN (unpaired image-to-image translation — horses to zebras), StyleGAN (style-based generation, progressive growing, high-resolution faces), Pix2Pix (paired image-to-image), BigGAN (large-scale, class-conditional ImageNet).
- Evaluation metric: FID (Frechet Inception Distance). Lower is better. Standard for comparing generative models. Students compute FID in the exercise.
- StyleGAN is mentioned for awareness — it produces the famous "this person does not exist" images.
- "If experts look bored": "FID measures the Frechet distance between Inception-v3 feature distributions of real and generated samples. It correlates with human quality judgments better than Inception Score, but it still has known failure modes — it can miss mode dropping and reward texture artefacts."

Transition: "Diffusion is the current state of the art."

Slide 59: Diffusion Models (Brief)

Time: ~3 min **Talking points**:

- DDPM (Denoising Diffusion Probabilistic Models): gradually add Gaussian noise to an image over T steps until it becomes pure noise. Then train a network to REVERSE that process — predict the noise at each step so you can subtract it out.
- Generation: start from pure noise, apply the trained denoising network iteratively until you recover a clean sample.
- Advantages over GANs: more stable training, better diversity, no mode collapse.
- Disadvantages: slow generation (many forward passes), compute-intensive training.
- Stable Diffusion is the practical application students have likely encountered — text-to-image generation deployed at massive scale.
- Students do not implement diffusion models in this module (too compute-intensive for a single exercise), but they should know the category.
- "If beginners look confused": "Start from noise and clean it up step by step. Each step the network says 'I think this pixel should be a little less noisy like this.' After enough steps, you have an image."
- "If experts look bored": "The connection to score matching (Song and Ermon) and continuous-time stochastic differential equations gives the field its mathematical depth. Classifier-free guidance is the key trick that made conditional diffusion work for text-to-image."

Transition: "Training GANs is notoriously hard. Let us name the failure modes."

Slide 60: GAN Training Challenges

Time: ~2 min **Talking points**:

- Mode collapse: the generator produces only a few output modes, ignoring the diversity of the real distribution. Classic symptom: every fake looks the same.
- Training instability: loss oscillates, sudden quality drops, non-convergence.
- Evaluation difficulty: loss values do not correspond to image quality in vanilla GAN (but they do in WGAN).
- Mitigations: WGAN-GP (most stable default), spectral normalisation, one-sided label smoothing, balanced update ratios.
- "Use WGAN-GP as your default for new projects. It is the most stable variant and the training dynamics are predictable."
- "If beginners look confused": "GAN training is more like a weather system than a deterministic optimisation. Expect oscillation. Monitor sample quality, not just loss numbers."

Transition: "Exercise time."

Slide 61: Exercise 5.5 — Generative Models

Time: ~1 min **Talking points**:

- Three tasks: implement DCGAN on a small image dataset, implement WGAN-GP and compare training stability visually, compute FID for both.
- Key comparison: WGAN-GP loss decreases smoothly and correlates with quality; DCGAN loss oscillates and does not correlate. Students should see this in their own training curves.

- Discuss when to use GANs vs diffusion vs VAE: images -> GAN or diffusion, text -> transformers, time-series -> VAE or LSTM.

Transition: "Lesson summary."

Slide 62: Lesson 5.5 Summary

Time: ~1 min **Talking points**:

- GANs learn to generate by adversarial competition. WGAN stabilises training with Wasserstein distance. Diffusion models are the current SOTA by noising and denoising. FID is the standard evaluation metric.
- Bridge: we have covered grids (CNN), sequences (RNN, Transformer), and generation (VAE, GAN, diffusion). The next data structure is graphs — irregular, connected, variable-size.

Transition: "Into the graph world."

Slide 63: Lesson 5.6 — Graph Neural Networks (Title)

Time: ~1 min **Talking points**:

- Title the lesson. Graphs are everywhere: social networks, molecular structures, knowledge graphs, citation networks, road networks.
- The key insight: a node's representation depends on its neighbours. GNNs generalise deep learning to non-Euclidean data.
- "If beginners look confused": "Whenever your data has connections — people who know people, atoms bonded to atoms, web pages linked to web pages — that is a graph. GNNs are how we learn from it."

Transition: "Start with the vocabulary."

Slide 64: Graph Data — Nodes, Edges, Adjacency

Time: ~2 min **Talking points**:

- Graph G = (V, E). V is the set of nodes (vertices). E is the set of edges (connections between nodes).
- Adjacency matrix A: element A_ij is 1 if there is an edge between node i and j, 0 otherwise. For large sparse graphs, stored as an edge list.
- Node features X: each node has a feature vector. For a social network, this might be user demographics. For molecules, atom types.
- Combine A and X to produce learned representations of each node.
- "If beginners look confused": "A graph is just a table of connections plus a table of node properties. The model learns to smear information along the connections."

Transition: "The GCN operation."

Slide 65: GCN — Graph Convolutional Networks

Time: ~4 min **Talking points**:

- Equation: H^(l+1) = sigma(D^(-1/2) A D^(-1/2) H^(l) W^(l)). Looks scary, intuition is simple.
- Walk through: H^(l) is the layer-l node features. A is the adjacency matrix. D is the degree matrix. D^(-1/2) A D^(-1/2) is symmetric normalisation.
- Plain English: for each node, average your neighbours' features (normalised by node degree), multiply by a learnable weight matrix, apply activation.
- The normalisation is the spectral theory connection — it is the symmetric normalised Laplacian from graph signal processing. You do not need the math; you need the intuition: "smoothed average of neighbours."
- After L layers, a node's representation reflects its L-hop neighbourhood. 2-3 layers is usually sufficient. More layers cause oversmoothing — all nodes become indistinguishable.
- "If beginners look confused": "Each layer, every node looks at its neighbours, averages their features, transforms, and updates. Do that twice and each node has seen information from 2 hops away."
- "If experts look bored": "The connection to spectral graph theory is via the graph Laplacian L = I - D^(-1/2) A D^(-1/2). The normalisation prevents feature magnitudes from exploding for high-degree nodes and supports the Chebyshev polynomial approximation from the original Kipf and Welling paper."

Transition: "Two important variants."

Slide 66: GraphSAGE and GAT

Time: ~3 min **Talking points**:

- GraphSAGE (Sample and Aggregate): instead of using the full neighbourhood, SAMPLE a fixed number of neighbours at each layer. This makes GNNs scalable to very large graphs. Also supports inductive learning — can handle unseen nodes at inference time (unlike transductive GCN).
- GAT (Graph Attention Network): learn attention weights over neighbours. Different neighbours get different importance. Equation: e_ij = LeakyReLU(a^T [Wh_i || Wh_j]), then softmax over j.
- GCN treats all neighbours equally. GAT learns to weight them. GraphSAGE makes them scalable.
- These three cover the core GNN design space: spectral (GCN), sampling (GraphSAGE), attention (GAT).
- "If beginners look confused": "GCN is 'average your friends.' GraphSAGE is 'average a random subset of your friends to handle big networks.' GAT is 'weight your friends by how similar they are to you.'"
- "If experts look bored": "GAT connects directly back to Lesson 5.4 — it is essentially self-attention restricted to a graph's edge structure. GIN (Graph Isomorphism Networks) is another important variant that achieves theoretical maximum expressiveness for distinguishing graph structures."

Transition: "What can GNNs actually predict?"

Slide 67: GNN Task Types

Time: ~2 min **Talking points**:

- Three task levels. Node classification: predict a label for each node (e.g. fraud detection, topic classification of research papers). Graph classification: predict a label for a whole graph (e.g. toxicity of a molecule). Link prediction: predict whether an edge should exist (e.g. recommender systems).
- Each task needs a different head on top of the GNN body. For graph classification, you aggregate all node embeddings with a readout function (sum, mean, max pooling). For link prediction, you score pairs of node embeddings.
- "If beginners look confused": "Same GNN body, different heads, different tasks. Like the classification vs regression split in M3."

Transition: "Exercise time."

Slide 68: Exercise 5.6 — Graph Neural Networks

Time: ~1 min **Talking points**:

- Task: graph classification on TUDataset using torch_geometric. Compare GCN vs GAT. Visualise attention weights and learned node embeddings.
- torch_geometric handles the graph data loading and batching. You focus on building the architecture.
- The attention weight visualisation is the highlight — it shows which molecular bonds or social connections the model considers important. Interpretability for free.
- Note: exercise uses PyTorch Geometric (built on PyTorch), not Kailash-specific tooling.

Transition: "Lesson summary."

Slide 69: Lesson 5.6 Summary

Time: ~1 min **Talking points**:

- GNNs extend deep learning to graph-structured data through message passing. GCN averages neighbours. GraphSAGE samples them. GAT attends over them. Three task types: node, graph, link.
- Bridge: after six lessons building architectures from scratch, students are ready for the practical shortcut — start with pre-trained models and fine-tune.

Transition: "The most practical technique in modern DL."

Slide 70: Lesson 5.7 — Transfer Learning (Title)

Time: ~1 min **Talking points**:

- Title the lesson. Transfer learning is the technique almost every practitioner uses every day. Very few professionals train from scratch anymore.
- Pre-trained models on ImageNet (vision) and large text corpora (language) provide universal features that transfer to almost any downstream task.
- "If beginners look confused": "For the rest of your career, this is how you will build models for new problems. Start with someone else's pre-trained model. Swap the head. Fine-tune. Done."

Transition: "The paradigm."

Slide 71: The Transfer Learning Paradigm

Time: ~2 min **Talking points**:

- Pre-training: train a large model on a massive dataset (ImageNet, The Pile, etc.). Learns general features — edges, textures, grammar, semantics.
- Fine-tuning: take those features, attach a new classifier head, train on your small target dataset. Much faster, much less data needed.
- Why it works: early layers learn universal features that transfer across tasks. Later layers learn task-specific features that you replace.

- The smaller your dataset, the more you benefit from transfer learning. Below 10k examples, transfer learning is essentially required.
- "If beginners look confused": "You do not have ImageNet-scale data. Someone else does. Use their trained model as the starting point for yours."
- "If experts look bored": "The scaling laws literature makes this explicit: pretrained compute transfers logarithmically to downstream tasks. More pretraining compute means better downstream, with diminishing returns."

Transition: "Let us fine-tune ResNet for computer vision."

Slide 72: CV Transfer Learning — ResNet Fine-Tuning

Time: ~3 min **Talking points**:

- Walk through the code: load pretrained ResNet from torchvision, freeze the early layers (requires_grad = False), replace the final fully connected layer with a new classifier for your number of classes, train with a LOW learning rate (1e-4 or lower).
- Key decisions: which layers to freeze, what learning rate to use, how much data augmentation. Progressive unfreezing is the safest strategy — start with only the head, then unfreeze one block at a time.
- Low learning rate is critical — too high and you destroy the pre-trained features.
- Data augmentation matters more for small target datasets — random crop, flip, colour jitter, mixup.
- Real applications: mask detection, medical imaging, satellite imagery.
- "If beginners look confused": "The recipe is: load a model, freeze most of it, swap the last layer, train at a very low learning rate. Three lines of PyTorch, massive accuracy improvement over training from scratch."
- "If experts look bored": "The 'which layers to freeze' question is task-dependent. For very dissimilar domains (satellite imagery vs natural images), you unfreeze more layers. For similar domains, keep most frozen."

Transition: "Now the NLP side."

Slide 73: NLP Transfer Learning — BERT Fine-Tuning

Time: ~3 min **Talking points**:

- Same pattern as CV but with BERT. Use HuggingFace Transformers: load a pre-trained BERT checkpoint, add a classification head, fine-tune on your labelled text data.
- Key hyperparameters: learning rate (2e-5 to 5e-5 is standard), number of epochs (3-5 is typical), batch size (16-32).
- Lightning-based training loop wraps the standard PyTorch fit/validate pattern.
- Mention adapter modules as a preview of M6: instead of fine-tuning all weights, insert small bottleneck layers between transformer blocks and only train those. This is the idea behind LoRA and parameter-efficient fine-tuning.
- "If beginners look confused": "Same pattern as ResNet — load pretrained, swap the head, fine-tune low-LR. HuggingFace makes it three lines."
- "If experts look bored": "BERT fine-tuning is being displaced by zero-shot and few-shot prompting with larger decoder-only LLMs. But for specialised domains with labelled data, fine-tuning BERT-sized encoders is still the highest-accuracy-per-dollar option."

Transition: "One big table to internalise."

Slide 74: Architecture Selection Guide

Time: ~2 min **Talking points**:

- The table on the slide is a key reference for professional practice. Walk through every row:
- Images -> CNN or ViT. Always use transfer learning (ImageNet pretrained).
- Text -> Transformer. Always use transfer learning (BERT or GPT pretrained).
- Sequences -> LSTM or Transformer. Sometimes transfer (domain-specific pretraining helps).
- Graphs -> GNN. Rarely transfer — usually train for the specific task.
- Tabular data -> Gradient boosting (XGBoost, LightGBM). Never transfer. Train from scratch.
- The "tabular data resists transfer" point is important. Many professionals work with tabular data — the advice is to use XGBoost or LightGBM, not deep learning. This connects back to M4 supervised learning.
- "If beginners look confused": "Memorise this table. It will save you from months of wrong-architecture projects."

Transition: "Two Kailash engines together — OnnxBridge and InferenceServer."

Slide 75: Kailash Bridge — OnnxBridge + InferenceServer

Time: ~3 min **Talking points**:

- OnnxBridge exports. InferenceServer serves. Together they are the deployment story.
- Export: OnnxBridge takes a trained PyTorch model and writes an ONNX file with validation.
- Serve: InferenceServer loads the ONNX file, exposes predict (single), predict_batch (many), warm_cache (pre-load for fast first prediction). Returns a structured PredictionResult object, not raw numpy arrays.
- The warm_cache call matters in production — it pre-loads the model so the first real prediction is as fast as the hundredth. Avoids the cold-start problem.
- Pattern: train anything in PyTorch, export with OnnxBridge, serve with InferenceServer. Works for any model trained in this module — CNN, transformer, GNN, all of them.
- "If beginners look confused": "Two functions. Export(). Serve(). You do not need to understand the internals. You just need to know when to call each."
- "If experts look bored": "The PredictionResult object carries metadata — model version, prediction timestamp, input schema, confidence scores. It is the structured alternative to the numpy-arrays-everywhere antipattern."

Transition: "The exercise."

Slide 76: Exercise 5.7 — Transfer Learning Pipeline

Time: ~1 min **Talking points**:

- Most practical exercise in the module. Two tasks: fine-tune ResNet for mask detection, fine-tune BERT for text classification. For each: export to ONNX with OnnxBridge, deploy with InferenceServer, serve at least one prediction through predict() and one through predict_batch().
- Comparison: fine-tuned vs trained-from-scratch baseline. Transfer should win convincingly, especially on small datasets.
- Note: exercise uses PyTorch + HuggingFace, wrapped with Kailash engines for the production pipeline.

Transition: "Lesson summary."

Slide 77: Lesson 5.7 Summary

Time: ~1 min **Talking points**:

- Transfer learning takes pre-trained models and fine-tunes them on target tasks. Low learning rate, partial freezing, data augmentation. ResNet for CV, BERT for NLP. OnnxBridge exports, InferenceServer serves.
- Bridge: all seven lessons so far learn from STATIC data (images, text, sequences, graphs). The last lesson is different. RL learns from INTERACTION with an environment.

Transition: "The final paradigm."

Slide 78: Lesson 5.8 — Reinforcement Learning (Title)

Time: ~1 min **Talking points**:

- Title the lesson. RL is the final paradigm in this module. It completes the DL picture: supervised (labelled data), unsupervised (structure discovery), generative (creating data), and now RL (learning from interaction).
- This lesson connects directly to RLHF for LLM alignment in M6 — PPO, which we cover today, is the algorithm behind RLHF.
- "If beginners look confused": "All the models so far learned from a fixed dataset. An RL agent doesn't have a dataset — it has an environment it can interact with. It takes actions, sees rewards, and learns a strategy."

Transition: "The paradigm shift."

Slide 79: The RL Paradigm Shift

Time: ~2 min **Talking points**:

- Supervised learning: input X, output y, fixed dataset, minimise loss. Static. Independent samples.
- RL: no dataset. Agent interacts with environment, takes action, receives reward, transitions to new state. Dynamic. Sequential. Correlated samples. Delayed rewards.
- Four hard problems RL agents face that supervised models do not: delayed rewards (credit assignment), sparse rewards (most steps give no feedback), exploration vs exploitation (do I try something new or exploit what I know?), non-stationarity (the data distribution changes as the agent learns).
- "If beginners look confused": "Supervised learning is a student with a textbook. RL is a baby learning to walk — falling, getting up, trying again, with no manual."
- "If experts look bored": "The non-stationarity is what makes RL hard. The agent's policy changes, which changes the state distribution, which changes what it should learn. No convergence guarantees like supervised learning."

Transition: "The vocabulary of RL."

Slide 80: RL Fundamentals

Time: ~3 min **Talking points**:

- Five core concepts:

- Agent: the learner (neural network).
- Environment: everything outside the agent. Provides observations and rewards.
- State: the agent's knowledge of the environment at time t.
- Action: what the agent does.
- Reward: a scalar signal from the environment indicating how well the agent is doing.
- Episode: a sequence of (state, action, reward) steps from start until termination.
- Policy pi(a|s): a mapping from states to actions. Can be deterministic or stochastic.
- Value function V(s): expected cumulative reward starting from state s. Q(s, a) is the same for (state, action) pairs.
- "If beginners look confused": "Think of a game. The agent is the player, the environment is the game, the state is the screen, the action is the controller input, the reward is the score. The policy is your strategy. The value function is how good you think a given screen is."

Transition: "The equations that make RL possible."

Slide 81: Bellman Equations

Time: ~4 min **Talking points**:

- Bellman expectation: V(s) = E[R_{t+1} + gamma * V(S_{t+1}) | S_t = s]. The value of a state is the immediate reward plus the discounted value of the next state. Recursive.
- Bellman optimality: Q*(s, a) = E[R_{t+1} + gamma * max_{a'} Q*(S_{t+1}, a')]. The max over next actions makes it OPTIMAL — the agent picks the best future action.
- Gamma is the discount factor (typically 0.9 to 0.99). Higher gamma = more forward-looking. Lower = more myopic.
- Why it matters: the recursive definition is what makes dynamic programming and Q-learning possible. If you can estimate Q* well, the optimal policy is just argmax_a Q*(s, a).
- "If beginners look confused": "The value of RIGHT NOW is the reward you get now plus a shrunken version of the value of the next moment. Do the same thing at the next moment, and you get a recursion that covers the whole future."
- "If experts look bored": "Bellman equations underlie all of dynamic programming, policy iteration, value iteration, and modern deep RL. The max in the optimality equation is what makes Q-learning off-policy — you can learn about the optimal policy while following a different behaviour policy."

Transition: "The first deep RL algorithm — DQN."

Slide 82: DQN — Deep Q-Network

Time: ~4 min **Talking points**:

- DQN is the entry point for deep RL. It uses a neural network to approximate Q(s, a). Trained to minimise the Bellman loss: L = E[(r + gamma * max_{a'} Q(s', a'; theta^-) - Q(s, a; theta))^2].
- Three innovations that made it work:
- Experience replay: store transitions in a buffer, sample random minibatches for training. Breaks correlation in sequential data.
- Target network: a separate, slowly updated copy of Q for computing the target r + gamma * max Q(s', a'; theta^-). Stabilises training.
- Epsilon-greedy: with probability epsilon take a random action, else take argmax Q. Balances exploration and exploitation.
- Application: customer churn prevention. The action is "intervene or not." The state is customer history. The reward is retention.

- "If beginners look confused": "DQN is Q-learning plus a neural network plus two tricks to keep it stable. Pick the action with the highest Q-value most of the time, explore sometimes."
- "If experts look bored": "The 2015 DeepMind Atari paper is the historical landmark. Every subsequent deep RL algorithm — DDPG, SAC, PPO — addresses limitations of DQN."

Transition: "DQN handles discrete actions. What about continuous?"

Slide 83: Policy Gradient Methods

Time: ~3 min **Talking points**:

- DQN works for DISCRETE actions (choose from a menu). But continuous actions (how much to adjust temperature, how much discount to offer) need a different approach.
- Policy gradient methods learn the policy pi(a|s) directly. The gradient of expected return with respect to policy parameters is estimated from rollouts.
- Actor-critic architecture: the actor learns the policy (what to do), the critic learns the value function (how good it is). The critic's estimate reduces variance in the policy gradient update.
- "If beginners look confused": "Instead of learning 'how good is each action,' learn 'what action should I take' directly. Then a second network helps reduce noise in the updates."
- "If experts look bored": "Policy gradient via REINFORCE has high variance. Actor-critic subtracts a baseline (the critic's V(s)) which dramatically reduces variance without biasing the estimator. The advantage A(s, a) = Q(s, a) - V(s) is the key quantity."

Transition: "Five algorithms in one overview slide."

Slide 84: Five Algorithms, Five Applications

Time: ~2 min **Talking points**:

- DQN — customer churn prevention (discrete actions, intervene or not).
- DDPG — manufacturing control (continuous actions, set machine parameters).
- SAC — dynamic pricing (continuous actions, handles uncertainty, entropy regularisation).
- A2C — resource allocation (discrete or continuous, variance reduction via baseline).
- PPO — supply chain optimisation (clipped objective, prevents destructively large updates).
- Students implement DQN and PPO in the exercise. The other three are covered conceptually.
- "Pick the algorithm by action space and stability requirements. Discrete: DQN or PPO. Continuous: SAC or PPO. Very unstable environment: PPO."

Transition: "Two continuous-action algorithms."

Slide 85: DDPG and SAC

Time: ~3 min **Talking points**:

- DDPG (Deep Deterministic Policy Gradient): extends DQN to continuous actions. Deterministic policy mu(s). Actor-critic. Off-policy (can reuse old experience via replay buffer).
- SAC (Soft Actor-Critic): adds entropy regularisation to the objective. Encourages exploration by preferring policies that are stochastic unless the environment rewards determinism.
- Both are off-policy, both scale well to high-dimensional continuous action spaces.

- SAC is generally preferred in modern practice — more stable than DDPG, less hyperparameter sensitive.
- "If beginners look confused": "DDPG is continuous-action DQN. SAC is DDPG plus 'stay exploratory' as part of the loss. Both are off-policy, which means they can train from a replay buffer."
- "If experts look bored": "SAC's entropy coefficient alpha is auto-tuned in modern implementations, which removes the main hyperparameter pain point. This is the default choice for continuous robotics benchmarks."

Transition: "A simpler actor-critic."

Slide 86: A2C — Advantage Actor-Critic

Time: ~2 min **Talking points**:

- A2C is the simplest actor-critic algorithm. Synchronous version of A3C (Asynchronous Advantage Actor-Critic).
- Advantage function A(s, a) = Q(s, a) - V(s). Tells the actor how much better or worse an action was compared to the baseline.
- Reduces variance in the policy gradient. Training is more stable than pure REINFORCE.
- Can be on-policy (uses current rollouts only), which makes it simpler but less sample-efficient than off-policy methods.
- "If beginners look confused": "The advantage tells the actor 'this action was better than average, do more of it' or 'this was worse, do less.' Using the baseline makes training much smoother."

Transition: "And the flagship algorithm — PPO."

Slide 87: PPO — Proximal Policy Optimization

Time: ~4 min **Talking points**:

- PPO is the most important RL algorithm to know in 2026. Robust, stable, widely deployed, and the algorithm behind RLHF.
- Core idea: when updating the policy, do not move too far from the old policy in a single step. Too large a step can destroy everything the agent has learned.
- Clipped objective: L^CLIP = E[min(r_t * A_t, clip(r_t, 1 - epsilon, 1 + epsilon) * A_t)], where r_t = pi_new(als) / pi_old(als) is the probability ratio.
- The clipping achieves the trust-region constraint of TRPO (do not change the policy too much) with a simple min + clip operation. Much simpler to implement, similar empirical performance.
- The M6 connection: RLHF uses PPO to align LLMs with human preferences. DPO later achieves the same goal without the reward model. The chain is: Lesson 5.4 (transformers) -> Lesson 5.7 (transfer) -> Lesson 5.8 (PPO) -> Module 6 (alignment).
- "If beginners look confused": "Take small steps. Do not let the new policy stray too far from the old one. That is literally the whole algorithm."
- "If experts look bored": "PPO is a first-order approximation to TRPO. It trades theoretical optimality for simplicity and gets around 95% of the performance. The clipped objective is one of the most pragmatic tricks in modern RL."

Transition: "To use RL for business problems, you need a custom environment."

Slide 88: Custom Gymnasium Environments

Time: ~3 min **Talking points**:

- Custom environments are where RL becomes practical for business. The Gymnasium API is the standard (successor to OpenAI Gym).
- Required methods: reset() (start a new episode, return initial observation), step(action) (apply action, return next observation, reward, termination flag, info dict).
- Define the observation space, action space, and reward function.
- The reward function is the most important design decision. It defines what "success" means. Bad reward design = reward hacking. Good reward design = aligned behaviour.
- Examples you will build in the exercise: customer churn environment (state = customer features, action = intervention type, reward = retention + cost), supply chain environment (state = inventory levels, action = order quantity, reward = profit - holding costs - stockouts).
- "If beginners look confused": "Three functions: reset, step, and a reward formula. That is the whole environment. The hard part is designing the reward so the agent learns what you actually want."
- "If experts look bored": "Reward shaping is where most business RL projects live or die. Specification gaming is a real failure mode — the agent finds a way to maximise reward that violates the spirit of the task."

Transition: "Meet the last Kailash engine — RLTrainer."

Slide 89: Kailash Bridge — RLTrainer

Time: ~3 min **Talking points**:

- RLTrainer wraps the RL training loop with Kailash's standard interface. Supports all five algorithms (DQN, DDPG, SAC, A2C, PPO) with a unified API.
- Usage pattern: instantiate with an environment and algorithm name, call train(total_steps), call evaluate() to run the trained policy and report metrics.
- RLTrainer handles the replay buffer, target network updates, logging, and checkpointing. Students focus on the environment and reward function — not the training loop plumbing.
- Note: exercise uses RLTrainer layered on PyTorch-based RL implementations.
- "If beginners look confused": "You write the environment. RLTrainer handles everything else. Two lines of code go from 'I have an environment' to 'I have a trained agent.'"
- "If experts look bored": "The unified interface lets you swap algorithms with a one-line change. Useful for the classic 'try PPO, try SAC, see which one works better for your problem' workflow."

Transition: "The exercise."

Slide 90: Exercise 5.8 — Reinforcement Learning

Time: ~1 min **Talking points**:

- Two tasks: implement DQN for customer churn prevention (discrete action), implement PPO for supply chain optimisation (continuous action). For each: design the environment, define the reward function, train with RLTrainer, evaluate the learned policy, compare with a rule-based baseline.
- The RLHF bridge to M6 is critical. Students who understand PPO here will have a head start on understanding LLM alignment in M6.
- Creative part: designing reward functions that capture the business objective without gameable loopholes.

Transition: "Key formulas."

Slide 91: Lesson 5.8 Key Formulas

Time: ~1 min **Talking points**:

- Four equations: Bellman expectation, Bellman optimality, DQN loss, PPO clipped objective.
- Bellman equations are the foundation. DQN loss turns Bellman into a supervised learning problem. PPO clips the policy update for stability.
- "These four lines are the foundation of deep RL. Internalise them."

Transition: "Lesson summary."

Slide 92: Lesson 5.8 Summary

Time: ~1 min **Talking points**:

- RL learns from interaction, not fixed datasets. Bellman equations define optimal value functions. DQN uses neural Q-networks. PPO clips the policy update. Custom Gymnasium environments enable business applications. RLTrainer handles the loop.
- Bridge to M6: RL is the last building block. M6 combines everything — transformers (Lesson 5.4), transfer learning (Lesson 5.7), and RL (Lesson 5.8) — into RLHF for LLM alignment. The progression is complete.

Transition: "Module-level consolidation."

Slide 93: Module 5 — Complete Formula Reference

Time: ~2 min **Talking points**:

- This is the cheat sheet. Walk through quickly, connecting each formula to its lesson context.
- Autoencoder reconstruction loss. VAE ELBO. Reparameterisation trick. Conv output size. ResNet skip. SE block. LSTM forget gate. GRU update gate. Scaled dot-product attention. Positional encoding. GAN minimax. GCN propagation. Bellman equation. PPO clipped objective.
- "You should have all of these memorised or readily derivable by the end of the module. If any feel unfamiliar, that is a signal to revisit the corresponding lesson."

Transition: "And the decision tree for when to use what."

Slide 94: Architecture Decision Tree

Time: ~2 min **Talking points**:

- Practical summary for choosing architectures. Three questions:
- What data type? (Images -> CNN/ViT. Text -> Transformer. Sequences -> LSTM/Transformer. Graphs -> GNN. Tabular -> Gradient boosting, not DL.)
- What goal? (Classify -> Supervised. Generate -> VAE/GAN/Diffusion/Transformer. Control -> RL.)
- Transfer learning by default — unless you have a good reason not to.
- "In a new project, start here. Data type -> architecture family. Goal -> training paradigm. Then ask: is there a pretrained model I can fine-tune? The answer is almost always yes."

- "If beginners look confused": "Three questions, done. You can solve 90% of new problems by answering them."

Transition: "End of module assessment."

Slide 95: End of Module Assessment

Time: ~2 min **Talking points**:

- Project format: choose a problem in your domain, apply the architecture decision tree, train a model, evaluate it, export to ONNX with OnnxBridge, deploy with InferenceServer.
- Assessment rubric: (1) architecture choice is justified, (2) model is trained to reasonable convergence, (3) evaluation uses appropriate metrics, (4) deployment pipeline works end-to-end.
- Open-ended but structured. You pick the domain. The methodology is fixed.
- Quiz component covers the formula reference and architecture decision tree.
- "The project is where everything comes together. Pick a problem you actually care about. The methodology you apply is the methodology you will use for the rest of your career."

Transition: "Module complete."

Slide 96: Module 5 Complete

Time: ~1 min **Talking points**:

- Thank the class. Module 5 is the most technically dense in the programme.
- Recap the progression: autoencoders compress, CNNs see space, RNNs remember time, transformers attend, GANs generate, GNNs connect, transfer learning scales, RL interacts.
- "Students who have completed all 8 lessons and exercises have a solid foundation in every major deep learning paradigm. Module 6 builds on this: LLMs, alignment, governance, and production AI."
- Read the closing line: "Every major DL architecture. One paradigm per lesson. All implemented."
- "If beginners look confused": "You did it. You now know more deep learning than most people who call themselves data scientists. Take a moment."
- "If experts look bored": "You have seen the lineage. Every modern AI system is a composition of these building blocks. Module 6 is where the compositions get interesting."

Transition: "See you in Module 6 for LLMs and alignment."

End of speaker notes for Module 5 — Deep Learning and ML Mastery in Vision and Transfer Learning.

Timing summary: - Intro + overview (Slides 1-5): ~10 min - Lesson 5.1 Autoencoders (Slides 6-17): ~30 min - Lesson 5.2 CNNs (Slides 18-29): ~26 min - Lesson 5.3 RNNs (Slides 30-40): ~22 min - Lesson 5.4 Transformers (Slides 41-52): ~26 min (+2 min Q&A pause) - Lesson 5.5 GANs (Slides 53-62): ~20 min - Lesson 5.6 GNNs (Slides 63-69): ~14 min - Lesson 5.7 Transfer Learning (Slides 70-77): ~16 min - Lesson 5.8 RL (Slides 78-92): ~34 min - Module-level close (Slides 93-96): ~7 min

Realistic total including pauses and transitions: approximately 180-200 minutes. Instructors running short on time should compress Lesson 5.6 (GNNs) and the module close; instructors running long should trim the GAN variants survey (Slide 58) and the DDPG/SAC slide (Slide 85). The four highest-priority slides that must not be cut: 13 (reparameterisation), 22 (ResNet), 43-44 (scaled dot-product attention), 87 (PPO). These are the conceptual anchors of the module.